



UNIVERSIDADE FEDERAL DO MARANHÃO
INSTITUTO DE ENGENHARIAS BICT
BACHARELADO INTERDISCIPLINAR EM CIÊNCIA E TECNOLOGIA
2026.1

DISCIPLINA: LABORATÓRIO DE PROGRAMAÇÃO

PROF. DR. SOFIANE BEN EL HEDI LABIDI

TURNOS: MATUTINO

Análise de Redes Sociais em Diferentes Paradigmas de Programação

DISCENTES: Lucas Moraes Nascimento, Thiago Santos Corrêa, Yuri Aguiar Gonçalves, Guilherme Sameneses de Lima

SÃO LUÍS – MA

2026

RESUMO

Este trabalho apresenta o desenvolvimento de um sistema de análise de redes sociais implementado em três paradigmas distintos: Orientação a Objetos (Python), Programação Lógica (Prolog) e Programação Funcional (Common Lisp). O objetivo foi comparar como diferentes paradigmas solucionam o mesmo problema computacional utilizando uma base de dados idêntica, composta por usuários e conexões sociais. Foram implementadas funcionalidades de consulta de relacionamentos, cálculo de graus de separação, identificação de amigos em comum e recomendação de conexões. A análise evidenciou diferenças expressivas na modelagem do problema, na estrutura dos algoritmos, na manipulação de estado e na legibilidade do código.

Palavras-chave: Paradigmas de Programação; Python; Prolog; Lisp; Redes Sociais; Grafos.

1 INTRODUÇÃO

Os paradigmas de programação representam diferentes formas de pensar e estruturar soluções computacionais. Cada paradigma possui características próprias que influenciam diretamente a maneira como problemas são modelados e resolvidos. Neste trabalho foi desenvolvido um sistema de análise de redes sociais utilizando três abordagens distintas:

- Programação Orientada a Objetos (Python);
- Programação Lógica (Prolog);
- Programação Funcional (Common Lisp).

A proposta consistiu em implementar exatamente o mesmo problema em cada paradigma para possibilitar uma comparação justa entre as diferentes abordagens. O problema escolhido foi uma rede social composta por usuários conectados por relações de seguimento, semelhante ao funcionamento de plataformas modernas como Instagram, X (Twitter) e LinkedIn.

2 OBJETIVOS

2.1 Objetivo Geral

Analisar comparativamente três paradigmas de programação por meio da implementação de um sistema de análise de redes sociais.

2.2 Objetivos Específicos

- Modelar uma rede social utilizando grafos direcionados;
- Implementar consultas de relacionamentos;
- Calcular graus de separação entre usuários;
- Identificar amigos em comum;
- Desenvolver sistema de recomendação de conexões;
- Comparar desempenho, legibilidade e facilidade de implementação entre os paradigmas.

3 DESCRIÇÃO DO PROBLEMA

O sistema modela uma rede social onde usuários podem seguir outros usuários. As funcionalidades exigidas foram:

1. Identificar quem um usuário segue diretamente;

2. Determinar a distância social entre dois usuários;
3. Encontrar conexões em comum;
4. Recomendar novas conexões baseadas em amigos de amigos.

A mesma base de dados foi utilizada nas três implementações para garantir equivalência dos resultados.

4 MODELAGEM DA REDE

A rede social utilizada contém 10 usuários, 18 conexões direcionadas e 1 ciclo de relacionamento (Judy → Alice), o que permite testar corretamente algoritmos de busca em grafos, especialmente o controle de nós visitados.

Usuário	Segue
Alice	Bob, Carol, Dave, Eve
Bob	Carol, Eve, Grace
Carol	Dave, Frank, Heidi
Dave	Eve, Grace
Eve	Frank
Frank	Grace
Grace	Heidi
Heidi	Ivan
Ivan	Judy
Judy	Alice ← ciclo

5 IMPLEMENTAÇÃO EM PYTHON – ORIENTAÇÃO A OBJETOS

A implementação orientada a objetos foi organizada em três classes principais, cada uma com responsabilidades bem definidas:

5.1 Classe Usuario

Representa cada nó da rede. Responsável por armazenar seguidores e usuários seguidos, além de fornecer acesso controlado aos dados.

5.2 Classe Conexao

Representa a aresta direcionada entre dois usuários, permitindo futura expansão para atributos como data da conexão, nível de interação ou peso da relação.

5.3 Classe RedeSocial

Gerencia todos os usuários e centraliza a execução dos algoritmos e consultas da rede.

5.4 Busca por Graus de Separação (BFS)

O algoritmo Breadth-First Search (BFS) foi adotado para calcular a distância social entre dois usuários. A implementação utiliza uma fila (deque), um conjunto de visitados e possui complexidade $O(V + E)$.

5.5 Sistema de Recomendações

- Encontrar amigos dos amigos;
- Remover conexões já existentes;
- Contabilizar frequência das ocorrências;
- Ordenar por relevância.

6 IMPLEMENTAÇÃO EM PROLOG – PROGRAMAÇÃO LÓGICA

Em Prolog a rede é descrita por meio de fatos que formam uma base de conhecimento. O programador declara apenas as regras; o mecanismo de inferência realiza automaticamente a busca pelas soluções.

6.1 Estrutura de Fatos

```
segue(alice, bob).  
segue(alice, carol).  
segue(bob, grace).
```

6.2 Exemplo de Consulta

```
?- segue(alice, X).  
X = bob ;  
X = carol ;  
...
```

6.3 Regra: Amigos de Amigos

```
amigo_de_amigo(A, C) :-  
    segue(A, B),  
    segue(B, C),  
    A \= C,  
    \+ segue(A, C).
```

6.4 Backtracking

Uma das principais vantagens observadas foi o uso do backtracking automático, que permite explorar todas as possibilidades sem necessidade de estruturas de controle explícitas.

7 IMPLEMENTAÇÃO EM COMMON LISP – PROGRAMAÇÃO FUNCIONAL

Na abordagem funcional o grafo é tratado como uma estrutura imutável. Não há alterações de estado após sua criação, e todas as operações são realizadas por funções puras.

7.1 Pipeline de Recomendações

1. Coleta de amigos dos amigos (mapcan);
2. Filtragem de conexões inválidas (remove-if);
3. Contagem de ocorrências;
4. Ordenação dos resultados (sort).

7.2 Características Observadas

Vantagens:

- Ausência de efeitos colaterais;
- Código altamente modular;
- Menor risco de inconsistências.

Desvantagens:

- Sintaxe complexa com grande quantidade de parênteses;
- Curva de aprendizado elevada para iniciantes.

8 ALGORITMO BFS NOS TRÊS PARADIGMAS

Apesar das diferenças de implementação, todos os paradigmas executam o mesmo algoritmo BFS conceitualmente. A tabela abaixo resume as diferenças de abordagem:

Paradigma	Fila	Controle de Visitados	Controle de Fluxo
Python (OO)	deque mutável	Conjunto (set)	Laço iterativo
Prolog	Lista como fila	Lista de visitados	Recursão + inferência
Common Lisp	Recursão explícita	Passagem por parâmetro	Recursão + imutabilidade

9 RESULTADOS E DISCUSSÃO

9.1 Comparação dos Paradigmas

Aspecto	Python (OO)	Prolog	Common Lisp
Paradigma	Orientado a Objetos	Lógico	Funcional
Estado	Mutável	Declarativo	Imutável
Controle de Fluxo	Laços	Backtracking	Recursão
Estruturas Principais	Sets e Objetos	Fatos e Regras	Funções puras
Legibilidade	Alta	Muito Alta	Média
Curva de Aprendizado	Baixa	Alta	Alta
Recomendações	Loops e Contadores	Regras e Consultas	Pipeline funcional

9.2 Facilidades Encontradas

Python

- Estruturas prontas (set, deque);
- Fácil depuração;
- Boa legibilidade.

Prolog

- Consultas extremamente simples;
- Pouco código para expressar regras;
- Inferência automática.

Common Lisp

- Funções altamente reutilizáveis;
- Código sem efeitos colaterais;
- Composição elegante de operações.

9.3 Dificuldades Encontradas

Python

- Necessidade de sincronizar estruturas de dados manualmente.

Prolog

- Implementação de BFS mais trabalhosa do que nos outros paradigmas.

Common Lisp

- Grande quantidade de parênteses;
- Sintaxe pouco intuitiva para iniciantes.

10 CONCLUSÃO

O desenvolvimento do sistema de análise de redes sociais permitiu observar claramente como diferentes paradigmas influenciam a forma de resolver um mesmo problema.

Python apresentou maior facilidade de desenvolvimento e manutenção, sendo ideal para sistemas que exigem manipulação frequente de estado e evolução contínua.

Prolog mostrou-se extremamente expressivo para consultas sobre relacionamentos, tornando-se a solução mais natural para problemas baseados em grafos e inferência lógica.

Common Lisp destacou-se pela elegância funcional, modularidade e ausência de efeitos colaterais — características importantes em sistemas que exigem previsibilidade e confiabilidade.

Conclui-se que não existe um paradigma universalmente superior. Cada abordagem apresenta vantagens específicas para determinados tipos de problemas, reforçando a importância do conhecimento de múltiplos paradigmas na formação de um profissional de computação.

REFERÊNCIAS

COLMERAUER, A.; ROUSSEL, P. The Birth of Prolog. ACM SIGPLAN Notices, 1993.

CLOCKSIN, W. F.; MELLISH, C. S. Programming in Prolog. 5. ed. Springer, 2003.

MCCARTHY, J. Recursive Functions of Symbolic Expressions and Their Computation by Machine. Communications of the ACM, 1960.

GRAHAM, P. ANSI Common Lisp. Prentice Hall, 1996.

ABELSON, H.; SUSSMAN, G. J. Structure and Interpretation of Computer Programs. MIT Press, 1996.

VAN ROSSUM, G.; DRAKE, F. L. Python 3 Reference Manual. CreateSpace, 2009.

SEBESTA, R. W. Concepts of Programming Languages. 12. ed. Pearson, 2019.

TIOBE Index. Disponível em: <https://www.tiobe.com/tiobe-index/>. Acesso em: 2026.